KEMENTERIAN RISET, TEKNOLOGI, DAN PENDIDIKAN TINGGI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" YOGYAKARTA
JL. SWK 104 (Lingkar Utara) Condongcatur, Yogyakarta 55283 Telp.(0274)486188, 486733, Fax. 486400
Jl. Babarsari 2, Tambakbayan, Yogyakarta 55281 Telp.(0274)486911
Email : info@upnyk.ac.id. Homepage : http://www.upnyk.ac.id

KUIS MATA KULIAH PRAKTIKUM SEMESTER GASAL TA. 2026/2027
PRAKTIKUM IMPLEMENTASI STRUKTUR DATA
PERTEMUAN 07 - Binary Tree dan Traversal
KERJAKAN SESUAI NAMA FUNCTION PADA STUDENT.CPP

STUDY CASE

Pertemuan ini membahas BINARY TREE, yaitu rangkaian node yang bercabang-cabang seperti pohon terbalik, beserta TRAVERSAL-nya, yaitu cara mencetak seluruh isi tree dengan urutan tertentu. Seluruhnya ada 3 soal dan 5 fungsi yang harus Anda kerjakan di file *`student.cpp`*. Soal hanya menjelaskan HASIL yang harus tercapai, bukan langkah-langkah pengerjaannya. Menentukan caranya adalah bagian dari latihan ini.

Setiap soal ditulis dalam satu tabel. Quest berisi ceritanya, yaitu apa yang sedang terjadi dan apa yang diminta, beserta hal khusus yang ikut dinilai. Parameter/Variabel berisi arti setiap parameter, satu per satu. Gambaran berisi contoh nyata beserta hasil yang benar. Nama parameter dan nama field ditulis di antara tanda petik miring, misalnya *`root`* atau *`left`*.

FUNCTION YANG DINILAI		
`preorder`	Soal 1a — Traversal: Root - Left - Right	**20 Poin**
`inorder`	Soal 1b — Traversal: Left - Root - Right	**20 Poin**
`postorder`	Soal 1c — Traversal: Left - Right - Root	**20 Poin**
`clear`	Soal 2b — Membersihkan: membuang tree	**25 Poin**
`jumlahDaun`	Soal 3 — Soal cerita: menghitung daun	**15 Poin**

ISTILAH YANG DIPAKAI DI SELURUH SOAL	
`node`	Satu kotak penyimpanan. Isinya tiga hal: sebuah angka (*`data`*), alamat cabang KIRI (*`left`*), dan alamat cabang KANAN (*`right`*).
`tree`	Rangkaian node yang bercabang-cabang. Bentuknya seperti pohon terbalik: akarnya di atas, rantingnya menjulur ke bawah.
`root`	Alamat node PALING ATAS. Cukup dengan memegang *`root`*, seluruh isi tree bisa dicapai — karena tiap node menyimpan alamat kedua cabangnya.
`left`	Alamat cabang kiri sebuah node.

ISTILAH YANG DIPAKAI DI SELURUH SOAL	
`right`	Alamat cabang kanan sebuah node.
`NULL`	Alamat kosong. Cabang yang tidak ada bernilai *`NULL`*, dan *`root`* bernilai *`NULL`* berarti tree sedang KOSONG.
`daun`	Node yang TIDAK punya cabang sama sekali, yaitu *`left`* DAN *`right`*-nya sama-sama *`NULL`*.

Tree Contoh yang Dipakai di Hampir Semua Soal

Supaya mudah, seluruh contoh memakai tree yang sama:

```
            50          <- root
           /  \
         80    20
        /  \     \
      10    70    90
           /
         30
```

Bacanya begini: node 50 punya cabang kiri 80 dan cabang kanan 20. Node 80 punya cabang kiri 10 dan cabang kanan 70. Node 70 punya cabang kiri 30 dan tidak punya cabang kanan. Node 20 tidak punya cabang kiri, cabang kanannya 90. Node 10, 30, dan 90 tidak punya cabang sama sekali — merekalah daunnya.

Dalam bentuk kode, tree itu dibangun begini (ini sudah disediakan di *`main()`* di bagian paling bawah file, jadi tidak perlu Anda tulis lagi):

```
Node* root         = new Node{50, NULL, NULL};
root->left         = new Node{80, NULL, NULL};
root->right        = new Node{20, NULL, NULL};
root->left->left   = new Node{10, NULL, NULL};
root->left->right  = new Node{70, NULL, NULL};
root->right->right = new Node{90, NULL, NULL};
root->left->right->left = new Node{30, NULL, NULL};
```

PERHATIAN: angka di sebelah kiri TIDAK harus lebih kecil. Lihat saja 80 yang berada di kiri 50. Pada pertemuan ini *`left`* dan *`right`* cuma menyatakan POSISI, bukan urutan besar-kecil. Aturan urutan baru muncul di pertemuan berikutnya (Binary Search Tree).

Satu Ide untuk Kelima Fungsi — Rekursi

Perhatikan cabang kiri dari tree di atas:

```
      80
     /  \
   10    70
        /
      30
```

Bentuknya SAMA PERSIS dengan tree utuh: ada node paling atas, ada cabang kiri, ada cabang kanan. Bagian tree seperti ini disebut SUBTREE, dan sifat itulah yang membuat pekerjaan pada tree paling enak ditulis secara rekursif: kerjakan node yang sedang dipegang,

lalu minta cabang kiri dan cabang kanan mengerjakan pekerjaan yang sama untuk dirinya masing-masing.

Sebuah fungsi yang memanggil dirinya sendiri harus punya TITIK BERHENTI, kalau tidak ia akan memanggil dirinya terus-menerus sampai program berhenti tidak wajar. Pada tree, titik berhentinya adalah keadaan yang paling sederhana: subtree yang KOSONG, yaitu *`root == NULL`*.

1a. preorder (Root - Left - Right) **20 Poin**

<table>
<tr><td>Quest</td><td colspan="2">Seluruh isi tree harus dicetak ke layar, satu per satu, sampai habis. Urutan mencetaknya yang menentukan: pada preorder, sebuah node dicetak LEBIH DULU, baru sesudah itu kedua cabangnya dikerjakan.

Urutannya sering disingkat: Root - Left - Right. Perhatikan bahwa aturan itu berlaku di SETIAP node, bukan cuma di `root`. Waktu giliran node 80 tiba, node 80 juga dicetak dulu sebelum cabang kiri dan cabang kanannya dikerjakan.

Setiap node dicetak TEPAT SATU KALI. Tidak ada yang terlewat, tidak ada yang dicetak dua kali. Angka dipisahkan spasi. Spasi berlebih di akhir, atau pergantian baris di antara angka, tidak mengurangi nilai — yang dinilai urutan angkanya. Jangan mencetak apa pun selain angka dan spasi: tidak ada judul seperti "Preorder:", tidak ada koma, tidak ada tanda kurung. Tree kosong (`root` bernilai `NULL`) tidak mencetak apa pun, dan program tidak boleh berhenti tidak wajar. Mencetak tidak boleh mengubah tree. Memanggil fungsi ini dua kali berturut-turut harus mencetak hal yang sama persis.</td></tr>
<tr><td rowspan="2">Parameter/ Variabel</td><td><code>root</code></td><td>Alamat node paling atas tree yang mau dicetak. Boleh bernilai `NULL`, artinya tree sedang kosong</td></tr>
<tr><td><code>return</code></td><td>Tidak ada (fungsi bertipe `void`). Hasilnya dicetak langsung ke layar dengan `cout`, bukan dikembalikan</td></tr>
<tr><td>Gambaran</td><td colspan="2">
<pre>
        50
       / \
     80   20   preorder(root) mencetak:
    / \   \
  10   70   90   50 80 10 70 30 20 90
       /
     30

Runut   50 dicetak dulu (dia root), lalu giliran cabang kirinya;
          80 dicetak, lalu giliran cabang kirinya;
          10 dicetak, tidak punya cabang, selesai;
          70 dicetak, lalu 30 (kirinya), kanannya tidak ada;
          cabang kiri 50 sudah habis, sekarang cabang kanannya;
          20 dicetak, cabang kirinya tidak ada, lalu 90.

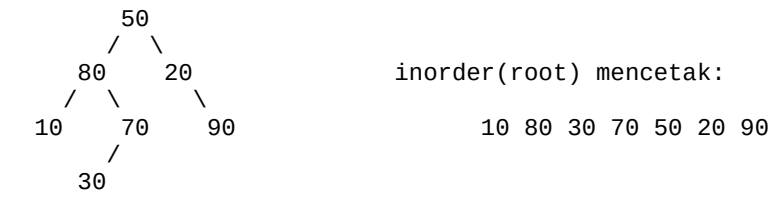
Sebelum : (kosong)
Operasi : preorder(NULL);
Sesudah : layar tidak bertambah apa-apa

Sebelum : Node satu{42, NULL, NULL};
Operasi : preorder(&satu);
Sesudah : mencetak 42
</pre>
</td></tr>
</table>

1b. inorder (Left - Root - Right) **20 Poin**

<table>
<tr><td>Quest</td><td colspan="2">Node-node yang dicetak sama persis dengan preorder, sama banyaknya pula. Yang berbeda hanya KAPAN sebuah node dicetak: pada inorder, cabang KIRI dikerjakan sampai habis lebih dulu, BARU node itu sendiri dicetak, baru kemudian cabang kanannya dikerjakan.

Urutannya sering disingkat: Left - Root - Right. Bandingkan dengan preorder yang mencetak 50 80 10 70 30 20 90. Angkanya sama, hanya urutannya yang berpindah. Yang paling kentara: 50 tadinya di paling depan, sekarang berada di TENGAH — karena seluruh cabang kirinya harus selesai lebih dulu.

Setiap node dicetak TEPAT SATU KALI. Angka dipisahkan spasi, tanpa judul, koma, atau tanda kurung. Tree kosong tidak mencetak apa pun, dan program tidak boleh berhenti tidak wajar. Mencetak tidak boleh mengubah tree. Hati-hati jangan sampai tertukar dengan postorder. Bedanya cuma satu baris posisinya, tetapi hasilnya berbeda jauh.</td></tr>
<tr><td rowspan="2">Parameter/ Variabel</td><td><code>root</code></td><td>Alamat node paling atas tree yang mau dicetak. Boleh bernilai `NULL`, artinya tree sedang kosong</td></tr>
<tr><td><code>return</code></td><td>Tidak ada (fungsi bertipe `void`). Hasilnya dicetak langsung ke layar dengan `cout`</td></tr>
<tr><td>Gambaran</td><td colspan="2"> 

Sebelum : (kosong)
Operasi : inorder(NULL);
Sesudah : tidak mencetak apa pun</td></tr>
</table>

1c. postorder (Left - Right - Root) **20 Poin**

<table>
<tr><td>Quest</td><td colspan="2">Sekali lagi node yang dicetak sama persis, dan sekali lagi yang berbeda hanya urutannya. Pada postorder, KEDUA cabang dikerjakan sampai habis lebih dulu, dan node itu sendiri dicetak PALING TERAKHIR.

Urutannya sering disingkat: Left - Right - Root. Akibatnya `root` selalu menjadi angka yang paling belakang. Sifat inilah yang nanti dipakai lagi pada Soal 2b: saat membuang tree, sebuah node baru boleh dibuang setelah kedua cabangnya selesai dibuang.

Setiap node dicetak TEPAT SATU KALI. Angka dipisahkan spasi, tanpa judul, koma, atau tanda kurung. Tree kosong tidak mencetak apa pun, dan program tidak boleh berhenti tidak wajar. Mencetak tidak boleh mengubah tree. `root` harus menjadi angka TERAKHIR yang dicetak. Kalau angka root Anda muncul di tengah, yang Anda tulis sebenarnya inorder.</td></tr>
<tr><td rowspan="2">Parameter/ Variabel</td><td><code>root</code></td><td>Alamat node paling atas tree yang mau dicetak. Boleh bernilai `NULL`, artinya tree sedang kosong</td></tr>
<tr><td><code>return</code></td><td>Tidak ada (fungsi bertipe `void`). Hasilnya dicetak langsung ke layar dengan `cout`</td></tr>
<tr><td>Gambaran</td><td colspan="2"> 

Ketiganya berdampingan supaya bedanya terlihat:

preorder : 50 80 10 70 30 20 90 <- 50 di depan
inorder : 10 80 30 70 50 20 90 <- 50 di tengah
postorder : 10 30 70 80 90 20 50 <- 50 di belakang

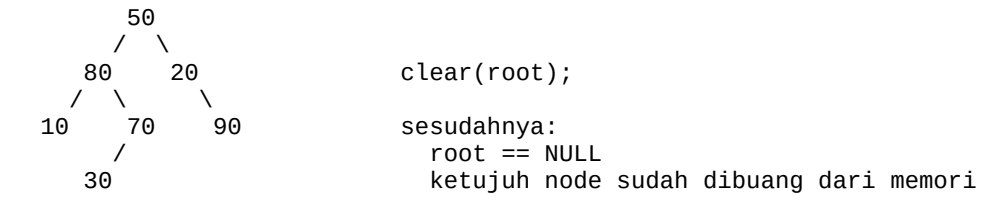
Sebelum : (kosong)
Operasi : postorder(NULL);
Sesudah : tidak mencetak apa pun</td></tr>
</table>

2b. clear (membersihkan tree) **25 Poin**

<table>
<tr><td>Quest</td><td colspan="2">Seluruh isi tree dibuang sekaligus, sampai tree benar-benar kosong. Bukan hanya diputus sambungannya — setiap node-nya harus dibuang dari memori satu per satu dengan `<i>delete</i>`.

Ada satu urutan yang tidak boleh dilanggar: sebuah node baru boleh dibuang SETELAH kedua cabangnya selesai dibuang. Kalau sebuah node dibuang lebih dulu, alamat kedua cabangnya ikut hilang bersama node itu — dan cabang-cabang tadi tertinggal di memori tanpa bisa dicapai lagi oleh siapa pun. Urutan "cabang dulu, baru node ini" itu persis urutan postorder pada Soal 1c.

Setelah selesai, `<i>root</i>` milik pemanggil harus bernilai `<i>NULL</i>`. SELURUH node harus dibuang dengan `<i>delete</i>`, masing-masing TEPAT SATU KALI. Banyaknya node yang dibuang ikut dihitung checker. Node yang cuma dilepas dari tree tetapi tidak di-`<i>delete</i>` akan tertinggal di memori, dan itu ikut dinilai. Membuang node yang SAMA dua kali adalah kesalahan berat: program biasanya langsung berhenti tidak wajar.

Hati-hati: alamat kedua cabang harus sudah dipakai SEBELUM sebuah node di-`<i>delete</i>`. Node yang sudah di-`<i>delete</i>` tidak boleh dibaca lagi, termasuk `<i>left</i>` dan `<i>right</i>`-nya. Memanggilnya pada tree yang sudah kosong, atau memanggilnya dua kali berturut-turut, harus aman dan tidak membuat program berhenti tidak wajar. Fungsi ini tidak mencetak apa pun ke layar.</td></tr>
<tr><td rowspan="2">Parameter/ Variabel</td><td><code>root</code></td><td>Alamat node paling atas tree milik pemanggil. Bertanda `&`, sehingga tree yang menjadi kosong ikut terasa oleh pemanggil. Boleh bernilai `<i>NULL</i>`, artinya tree memang sudah kosong</td></tr>
<tr><td><code>return</code></td><td>Tidak ada (fungsi bertipe `<i>void</i>`)</td></tr>
<tr><td>Gambaran</td><td colspan="2"> 

Sebelum : Node* root = NULL;
Operasi : clear(root);
Sesudah : aman, tidak terjadi apa-apa

Sebelum : root == NULL (clear sudah dipanggil sekali)
Operasi : clear(root);
Sesudah : aman, tidak terjadi apa-apa</td></tr>
</table>

3. jumlahDaun (soal cerita) **15 Poin**

<table>
<tr><td>Quest</td><td colspan="2">Sebuah tim kerja disusun berjenjang. Setiap orang membawahi paling banyak dua orang, dan setiap orang dicatat sebagai satu node dengan nomor pegawainya sebagai `data`. Orang yang dibawahi seseorang disimpan sebagai cabang `left` dan `right` miliknya.

Ketua tim ingin tahu satu hal: ada BERAPA ORANG yang tidak membawahi siapa-siapa. Merekalah yang bekerja langsung di lapangan, sementara yang lain sibuk mengurus bawahannya. Orang yang tidak membawahi siapa pun berarti node yang tidak punya cabang sama sekali — `left` DAN `right`-nya sama-sama `NULL`. Node seperti itu disebut DAUN.

Yang harus Anda putuskan sendiri: bagaimana mengenali sebuah daun, dan bagaimana menjumlahkan daun yang tersebar di kedua cabang. Perhatikan 20 dan 70 baik-baik. Keduanya cuma punya SATU cabang, dan itu tetap membuat mereka BUKAN daun. Daun harus benar-benar tidak punya cabang sama sekali.

Tree kosong hasilnya 0. Tree berisi satu node hasilnya 1. Node itu root sekaligus daun — kedua hal itu tidak saling meniadakan. Node yang punya SATU cabang saja BUKAN daun. Ini kesalahan yang paling sering terjadi pada soal ini. Daun bisa berada di kedalaman yang berbeda-beda. Semuanya tetap dihitung. Menghitung tidak boleh mengubah tree. Memanggil `jumlahDaun` dua kali berturut-turut harus memberi angka yang sama persis. Fungsi ini tidak mencetak apa pun ke layar. Ia hanya mengembalikan angka.</td></tr>
<tr><td rowspan="2">Parameter/ Variabel</td><td><code>root</code></td><td>Alamat node paling atas tim yang mau dihitung. Boleh bernilai `NULL`, artinya timnya sedang kosong</td></tr>
<tr><td><code>return</code></td><td>Banyaknya daun di dalam tree</td></tr>
<tr><td>Gambaran</td><td colspan="2">
<pre>
        50   jumlahDaun(root) -> 3
       / \
     80   20   Yang termasuk daun: 10, 30, dan 90.
    / \   \
  10   70   90   Yang BUKAN daun:
       /   50 punya dua cabang
     30   80 punya dua cabang
                              20 punya satu cabang (90)
                              70 punya satu cabang (30)

Sebelum : Node satu{42, NULL, NULL};
Operasi : jumlahDaun(&satu);
Sesudah : 1 (dia root sekaligus daun)

Sebelum : (kosong)
Operasi : jumlahDaun(NULL);
Sesudah : 0 (tidak ada orang sama sekali)

Tim yang menjulur lurus ke bawah hanya punya satu daun:

    5
   /
  4   jumlahDaun(root) -> 1
 /   (hanya node 3; node 5 dan 4 masing-masing
3   masih punya satu cabang)
</pre>
</td></tr>
</table>

CATATAN & ATURAN

- File yang boleh diedit hanya `*src/student.cpp*`. File yang tidak boleh diedit: `*src/student.h*`, `*tests/checker.cpp*`, dan `*tests/report.h*`.
- Signature fungsi dan bentuk `*struct Node*` adalah kontrak dan tidak boleh diubah. Isi fungsi, nama variabel, dan struktur kode di dalamnya sepenuhnya bebas.
- Anda boleh menambah fungsi bantu sendiri.
- Anda BOLEH menulis `*main()*` sendiri di file ini, di bagian paling bawah yang sudah disediakan. Pakai `*cin*` dan `*cout*` sebebasnya di sana. `*main()*` itu tidak ikut dinilai dan bebas Anda ubah sesuka hati.
- Setiap soal dinilai SENDIRI-SENDIRI. Checker membangun tree ujinya tanpa memakai fungsi Anda, jadi soal yang belum benar tidak ikut menjatuhkan nilai soal yang lain.
- `*cin*` hanya boleh dipakai DI DALAM `*main()*`. Jangan menaruh `*cin*` di dalam kelima fungsi yang dinilai, karena checker memanggil fungsi-fungsi itu tanpa memberi masukan apa pun, sehingga `*cin*` di sana akan membaca sampah dan nilai Anda berubah-ubah setiap kali dinilai.
- Kelima fungsi menerima datanya lewat PARAMETER. Itulah gunanya parameter: satu fungsi yang sama bisa dipakai berkali-kali dengan tree yang berbeda-beda.
- Untuk mencoba sendiri, tekan tombol Run di VS Code, atau jalankan `*g++ -std=c++17 src/student.cpp -o latihan && ./latihan*`. Saran: sebelum menjalankan program, TULIS DULU tebakan Anda untuk ketiga traversal di selembar kertas, baru jalankan dan bandingkan.
- Sebelum fungsi diisi, compiler memunculkan peringatan "unused parameter". Itu wajar dan tidak mengurangi nilai.